

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO BÀI TẬP LỚN
LẬP TRÌNH MẠNG

**Đề tài: Ironfront Reborn - Trò chơi bắn súng nhiều người chơi
với tầng mạng TCP/UDP tự xây dựng**

Lớp: D23CTPM01-B

Nhóm bài tập lớn: 10

Sinh viên thực hiện: Nguyễn Duy Nghĩa

Mã sinh viên: B23DCCN600

Báo cáo kiến trúc và giao diện hệ thống

**Phân cá nhân: *Phân hệ đồng bộ trạng thái và mô phỏng server*
*(Replication)***

Hà Nội - 2026

Mục lục

I. NỘI DUNG NHÓM	3
1. Mô tả kiến trúc chung của hệ thống	3
1.1. Bài toán và phạm vi	3
1.2. Kiến trúc ba tiến trình	3
1.3. Phân chia TCP và UDP theo đặc tính dữ liệu	5
1.4. Kiến trúc phân lớp hệ thống	5
1.5. Kiến trúc triển khai	6
1.6. Phân công theo phân hệ	7
2. Mô tả thiết kế chung của hệ thống	7
2.1. Mô hình thẩm quyền	7
2.2. Luồng nghiệp vụ đầu - cuối	8
2.3. Các kỹ thuật netcode cốt lõi	9
2.4. Thiết kế giao thức	9
2.5. Tham số thời gian	10
2.6. Phân loại đối tượng đồng bộ	11
2.7. Bảo mật ở mức phù hợp phạm vi	11
2.8. Các quyết định kiến trúc đã chốt	11
2.9. Quy trình phát triển và bảo đảm chất lượng	11
II. NỘI DUNG CÁ NHÂN - Phân hệ đồng bộ trạng thái và mô phỏng server (Replication).	13
3. Biểu đồ và giao diện nội dung cá nhân thực hiện	13
3.1. Kiến trúc phân hệ Replication	13
3.2. Vòng tick có thẩm quyền của server	14
3.3. Đường ống nén snapshot	14
3.4. Delta theo baseline đã xác nhận	15
3.5. Quản lý vùng quan tâm	15
3.6. Bán có thẩm quyền và bù trễ	16
3.7. Máy trạng thái trận đấu	17
4. Luồng xử lý và giải pháp đã ứng dụng	17
4.1. Luồng người chơi vào trận và spawn	18
4.2. Luồng input và vòng tick có thẩm quyền	19
4.3. Luồng tạo, nén và gửi snapshot	20
4.4. Luồng phân xử phát bắn với bù trễ	21
4.5. Luồng trận đấu, điểm chiếm cứ và báo kết quả	22
4.6. Luồng người chơi rời trận	23
4.7. Bảng ảnh xạ hình vẽ với chức năng cá nhân thực hiện	23

I. NỘI DUNG NHÓM

1. Mô tả kiến trúc chung của hệ thống

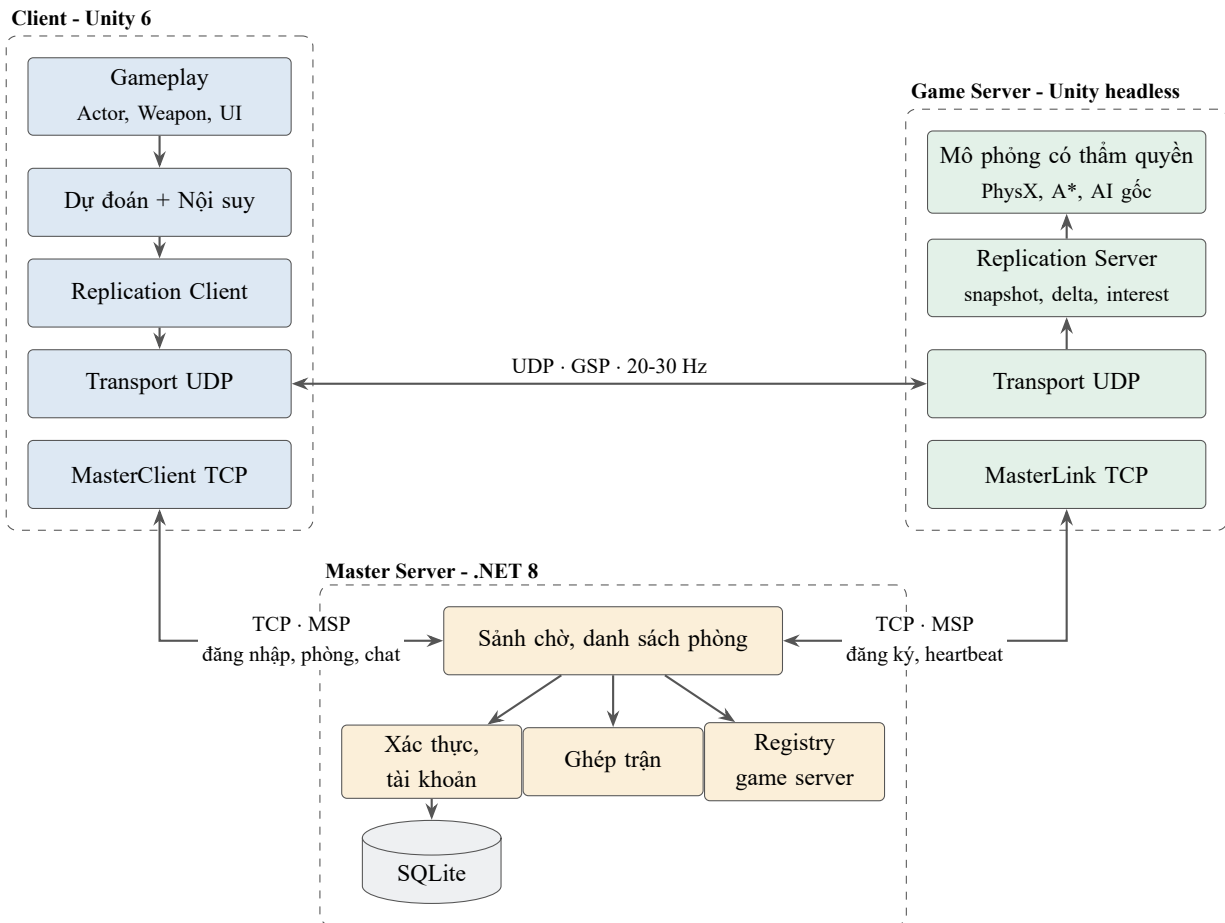
1.1. Bài toán và phạm vi

Ironfront Reborn chuyển mã nguồn trò chơi bắn súng góc nhìn thứ nhất *Ravenfield Beta 5* (Unity 6000.3, vốn chỉ chơi đơn) thành một trò chơi **hiều người chơi, máy chủ nắm quyền quyết định** (server-authoritative). Yêu cầu cốt lõi của môn Lập trình mạng: **toàn bộ tầng mạng được viết tay trên socket TCP và UDP thô** của hệ điều hành - không dùng Mirror, Photon, Netcode for GameObjects, KCP, gRPC hay WebSocket.

Tiêu chí	Giá trị thiết kế
Quy mô một trận	16 người chơi thật + 32 bot AI (tối đa 64 actor, 16 phương tiện)
Tần số mô phỏng / snapshot / gửi input	30 Hz / 20 Hz / 30 Hz
Băng thông xuống mỗi client	≤ 8 KB/s (đo trên server đã triển khai: 1,67 KB/s)
Triển khai	LAN trước, sau đó VPS công cộng (Docker Compose, TLS cho kênh TCP)
Công nghệ	.NET 8 (master server, công cụ), netstandard2.1 (thư viện dùng chung), Unity 6 (client và game server headless)

1.2. Kiến trúc ba tiến trình

Hệ thống gồm **ba loại tiến trình** độc lập, giao tiếp qua **hai giao thức tự định nghĩa**: **MSP** (Master Server Protocol, trên TCP) cho dịch vụ săn chờ và **GSP** (Game Server Protocol, trên UDP) cho lưu lượng trong trận. Hai giao thức gặp nhau ở đúng một điểm: **joinTicket** được master ký HMAC và game server tự kiểm tra.



Hình 1.1. Kiến trúc tổng thể ba tiến trình và hai giao thức MSP/GSP

Tiến trình	Nền tảng	Trách nhiệm chính	Giao tiếp
Client	Unity 6, bản build thường	Hiển thị, nhận input, dự đoán chuyển động người chơi cục bộ, nội suy actor ở xa, giao diện sảnh và HUD	TCP tới Master; UDP tới Game Server
Game Server	Unity 6 headless (chế độ không đồ họa)	Chạy mô phỏng có thẩm quyền (vật lý, AI, bắn, điểm chiếm), tạo snapshot riêng cho từng client, bù trễ	Nhận UDP từ client; TCP tới Master
Master Server	.NET 8 console, SQLite	Tài khoản, phiên, phòng, ghép trận, chat sảnh, registry game server, cấp joinTicket	Nhận TCP từ client và game server

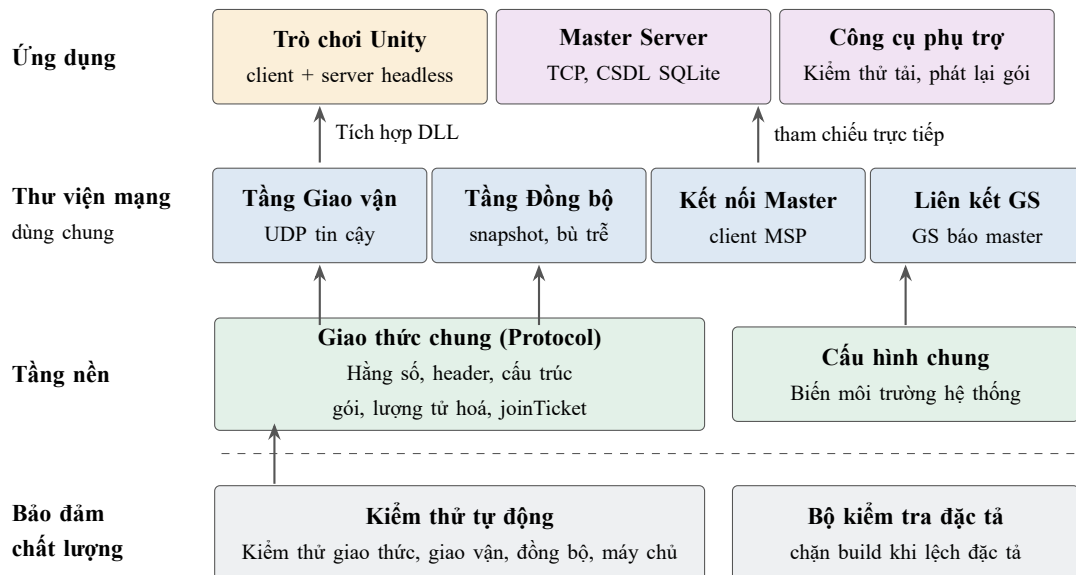
1.3. Phân chia TCP và UDP theo đặc tính dữ liệu

Đặc tính	TCP - Master Server	UDP - Game Server
Dùng cho	Đăng nhập, danh sách phòng, tạo/vào phòng, ghép trận, chat sảnh, heartbeat server	Mọi lưu lượng trong trận: input, snapshot, sự kiện gameplay
Tần suất	Vài gói/phút mỗi client	20-60 gói/giây mỗi chiều
Mất gói	Không chấp nhận (mất LOGIN_RES là treo màn hình)	Chấp nhận với trạng thái: snapshot sau thay thế snapshot trước
Nếu đổi chỗ	Phải tự viết lại độ tin cậy mà TCP đã làm tốt	Head-of-line blocking: một gói mất chặn mọi gói sau; Nagle + delayed ACK thêm 40-200 ms
Đóng khung	Tiền tố độ dài 4 byte + JSON	Datagram tự nhiên, header nhị phân 16 byte

Không dùng WebSocket: WebSocket chạy trên TCP nên thừa hưởng nguyên vẹn head-of-line blocking, cộng thêm chi phí bắt tay HTTP - một ràng buộc chỉ cần cho trình duyệt, không cần cho client desktop.

1.4. Kiến trúc phân lớp hệ thống

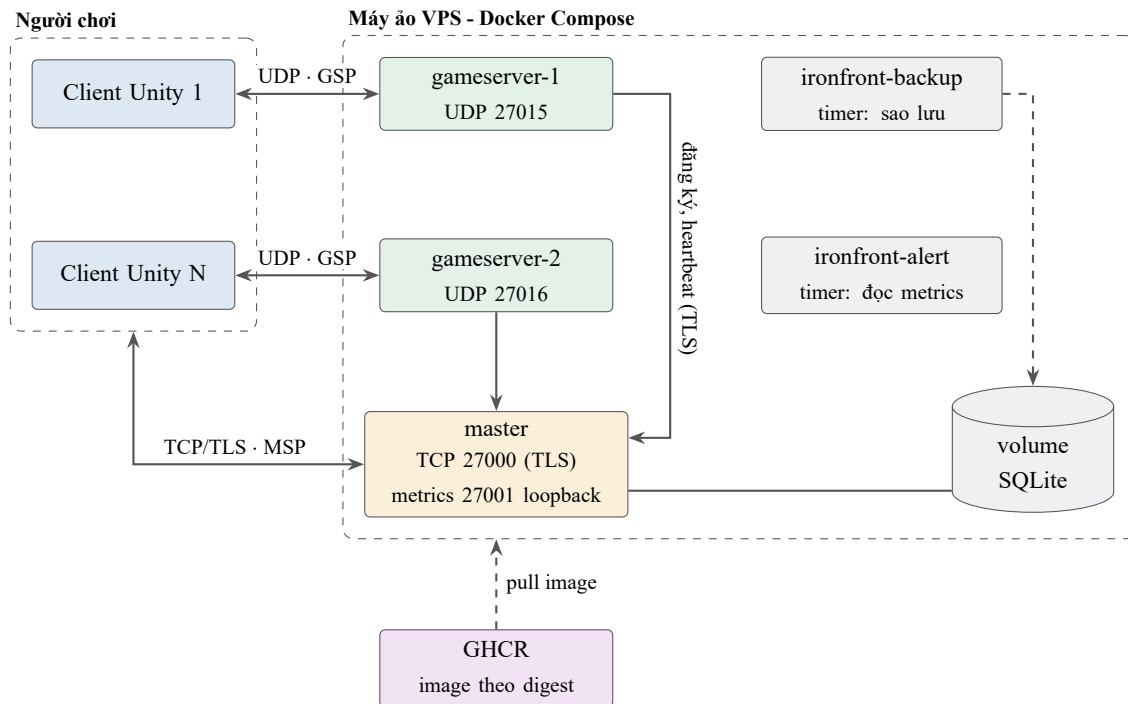
Mã nguồn chia thành các thư viện dùng chung độc lập hoàn toàn với engine đồ hoạ; dự án Unity tích hợp chúng dưới dạng các thư viện liên kết động (DLL). Nhờ vậy logic mạng được kiểm thử tự động độc lập nhanh chóng mà không cần mở Unity Editor.



Hình 1.2. Phân lớp kiến trúc và quan hệ phụ thuộc giữa các phân hệ

Phân hệ / Thành phần	Loại	Quy mô	Vai trò nghiệp vụ
Giao thức chung (Protocol)	Thư viện dùng chung	5 300 dòng	Định dạng gói tin, chuẩn tuần tự hoá và hằng số giao thức
Tầng Giao vận (Transport)	Thư viện giao vận	4 100 dòng	Độ tin cậy trên UDP, kênh truyền, phân mảnh, kiểm soát nghẽn
Đồng bộ và Mô phỏng	Thư viện logic	18 500 dòng	Snapshot, mã hóa delta, vùng quan tâm, tick server, bù trễ
Máy chủ trung tâm (Master)	Ứng dụng dịch vụ	4 300 dòng	Tiếp nhận TCP, xác thực, quản lý phòng, ghép trận, bảo mật
Liên kết máy chủ trung tâm	Thư viện kết nối	1 250 dòng	API trao đổi thông tin với máy chủ trung tâm
Tích hợp giao diện client	Tích hợp ứng dụng	14 600 dòng	Điều phối luồng, dự đoán, nội suy, giao diện sảnh và HUD

1.5. Kiến trúc triển khai



Hình 1.3. Sơ đồ triển khai trên một máy ảo: một master và hai game server headless

Image được triển khai theo bản dựng cố định; khóa bí mật dùng chung và chứng chỉ TLS được cấu hình qua biến môi trường bảo mật độc lập. Sao lưu và cảnh báo chạy bằng bộ định thời hệ thống *ngoài* container, vì bộ phát hiện sự cố phải nằm ngoài thứ nó giám sát. Hạ tầng còn có cấu hình tự động hoá máy ảo và phương án triển khai đám mây thay thế.

1.6. Phân công theo phân hệ

Sinh viên	Mã SV	Phân hệ	Phạm vi nhiệm vụ chính
Nguyễn Thành Trung	B23DCCN861	Client Unity	Giao diện sảnh, HUD, dự đoán và nội suy chuyển động client
Nguyễn Minh Toàn	B23DCCN832	Tầng giao vận UDP	Giao thức UDP tin cậy, kênh truyền, phân mảnh, ACK bitfield
Nguyễn Duy Nghĩa	B23DCCN600	Replication, mô phỏng server	Vòng tick mô phỏng, đồng bộ snapshot, nén delta và bù trễ
Nguyễn Tự Kiên	B23DCCN465	Master Server TCP, hạ tầng	Máy chủ TCP, xác thực, quản lý phòng, ghép trận và hạ tầng VPS

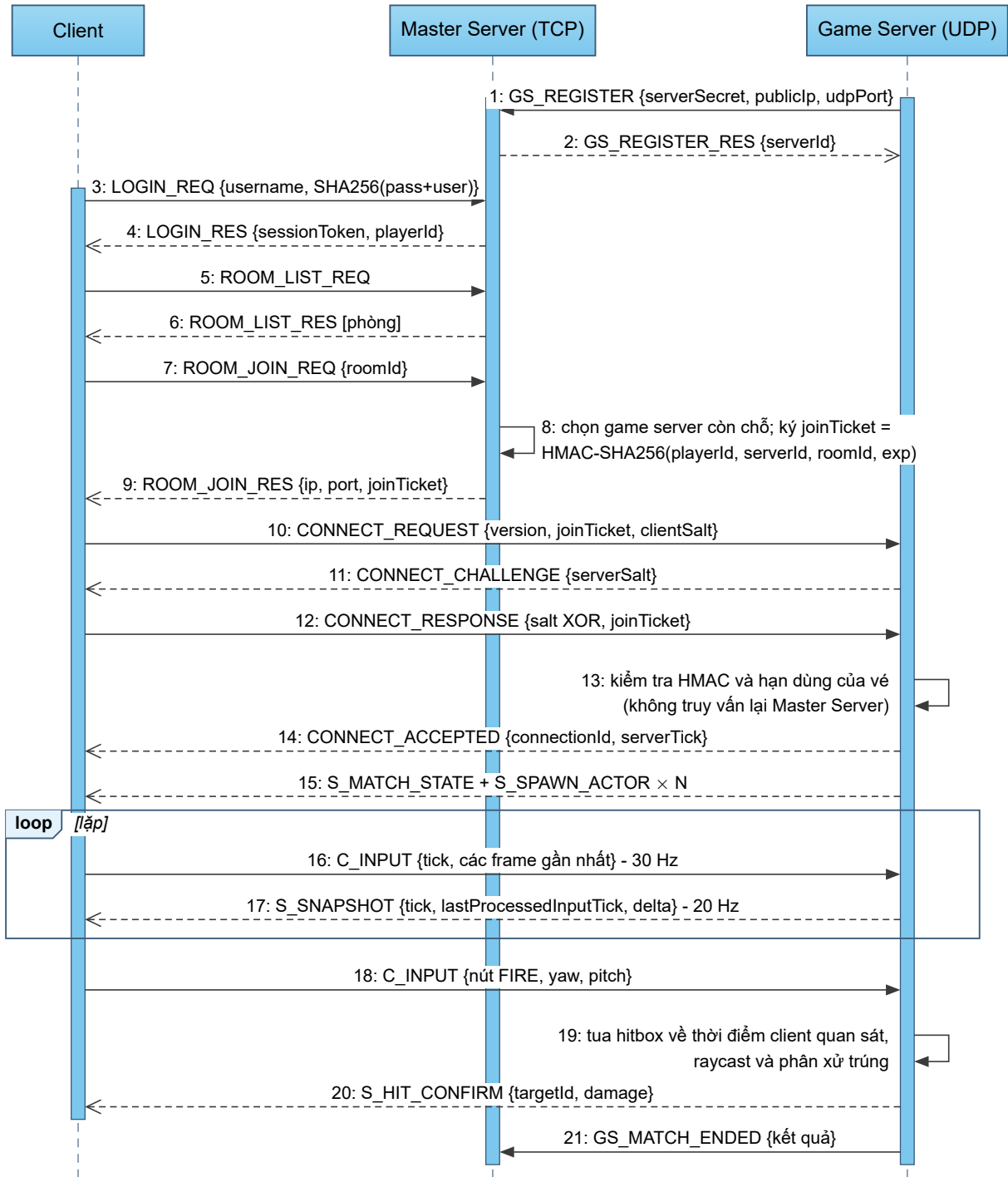
2. Mô tả thiết kế chung của hệ thống

2.1. Mô hình thẩm quyền

Hệ thống **hoàn toàn do server quyết định**: client chỉ gửi *ý định* (input), không bao giờ gửi kết quả. Nhờ vậy gian lận tốc độ, dịch chuyển tức thời, bắn nhanh hay đạn vô hạn trở nên bất khả thi về cấu trúc thay vì chỉ bị phát hiện.

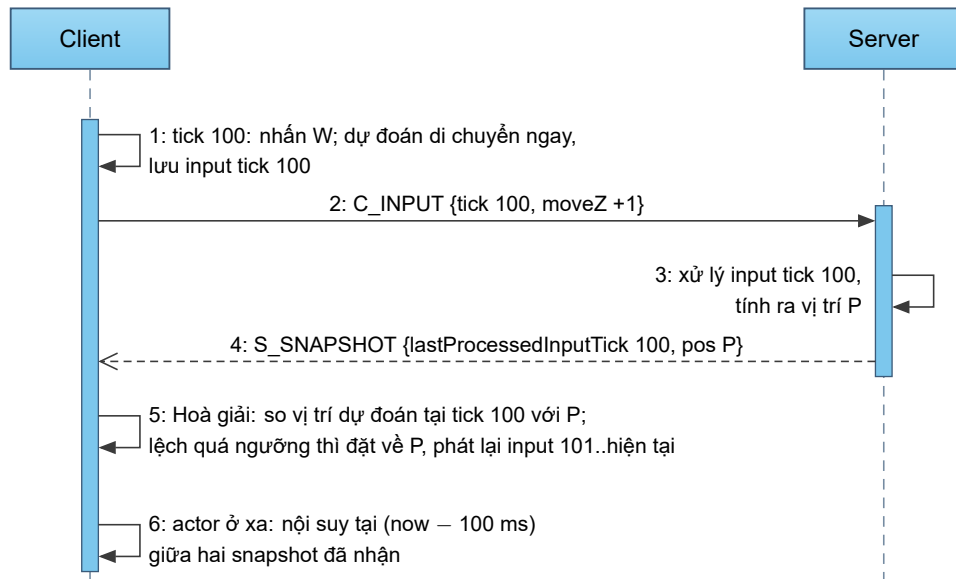
Vấn đề	AI quyết định	Client được làm gì
Vị trí nhân vật	Server	Dự đoán ngay, sau đó hoà giải với server
Trúng đạn, sát thương	Server	Hiện hiệu ứng dự đoán, chờ S_HIT_CONFIRM
Máu, chết, hồi sinh	Server	Chỉ hiển thị
Độ tàn đạn, giật súng	Server	Giật cục bộ cho cảm giác, không ảnh hưởng viên đạn thật
Bot AI, điểm chiếm, điểm số	Server	Nội suy, hiển thị
Lên xuống ghế phương tiện	Server	Gửi yêu cầu C_SEAT_REQUEST
Camera, UI, âm thanh, ragdoll	Client	Toàn quyền, không đồng bộ

2.2. Luồng nghiệp vụ đầu - cuối



Hình 2.1. Luồng đầu - cuối: đăng nhập, vào phòng, kết nối game server; chơi, kết thúc trận

2.3. Các kỹ thuật netcode cốt lõi

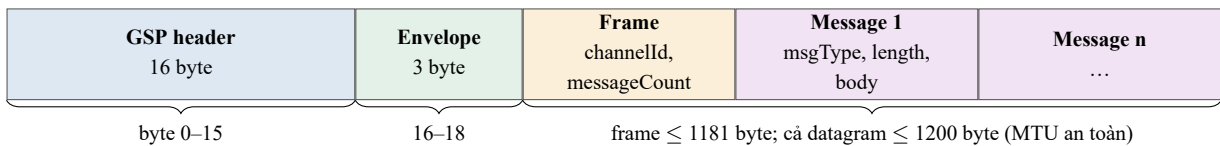


Hình 2.2. Dự đoán phía client, hoà giải với server và nội suy actor ở xa

Kỹ thuật	Mục đích	Tham số
Dự đoán phía client	Người chơi cục bộ di chuyển ngay khi nhấn phím	Cùng thuật toán mô phỏng chuyển động ở client và server
Hoà giải	Sửa sai khi dự đoán lệch, phát lại input chưa xác nhận	Ngưỡng lệch 0,25 m
Nội suy thực thể	Actor ở xa chuyển động mượt dù snapshot chỉ 20 Hz	Bộ đệm 100 ms
Bù trễ	Người ping cao vẫn bắn trúng thứ họ thấy	Tua tối đa 200 ms, lịch sử hitbox 1 s

2.4. Thiết kế giao thức

2.4.1. GSP - cấu trúc một datagram PAYLOAD



Hình 2.3. Ba lớp của một datagram PAYLOAD: header GSP, envelope kênh của transport, frame message của ứng dụng

Kênh	Kiểu	Dữ liệu	Khi mất gói
0	Không tin cậy, không thứ tự	Ping/pong	Bỏ qua
1	Không tin cậy, có thứ tự (server → client)	S_SNAPSHOT, S_WEAPON_FIRE, S_VEHICLE_SNAPSHOT	Bỏ qua; gói cũ bị hủy
2	Tin cậy, có thứ tự	Sự kiện gameplay, spawn/despawn, chat, C_ACK_BASELINE	Gửi lại tới khi được ack
3	Không tin cậy, có thứ tự (client → server)	C_INPUT, C_VEHICLE_INPUT	Bỏ qua; bù bằng dư thừa

Độ tin cậy áp dụng **theo kênh**, không theo kết nối: một sự kiện bị mất trên kênh 2 không làm nguyền snapshot trên kênh 1 - ưu điểm cốt lõi so với TCP vốn ép mọi thứ vào một luồng.

2.4.2. MSP - khung tin trên TCP

length u32, big-endian	msgType u16, little-endian	body UTF-8 JSON, tối đa 64 KB
----------------------------------	--------------------------------------	---

Hình 2.4. Khung tin MSP: tiền tố độ dài giải quyết bài toán ranh giới thông điệp trên luồng byte TCP

2.4.3. joinTicket - cầu nối TCP và UDP

playerId u32	serverId u16	roomId u16	expiresAtUnixMs u64 (60 s)	displayName 16 byte UTF-8	hmac HMAC-SHA256, 32 byte
32 byte dữ liệu được ký					32 byte chữ ký

Hình 2.5. Cấu trúc joinTicket 64 byte

Master ký vé bằng khoá bí mật; game server kiểm tra chữ ký bằng thuật toán so sánh thời gian hằng số và kiểm hạn dùng. Không cần gọi lại master, nên **master có thể dừng giữa trận mà trận vẫn tiếp tục**.

2.5. Tham số thời gian

Tham số	Giá trị	Ghi chú
Tick mô phỏng server	30 Hz (33,3 ms)	Theo đồng hồ thực, không theo chu kỳ cập nhật vật lý
Snapshot	20 Hz (50 ms)	Rẻ hơn 30 Hz, nội suy bù độ mượt
Gửi input	30 Hz	Mỗi gói lặp lại các frame gần nhất
Bộ đệm nội suy	100 ms	Chịu được mất một snapshot
Cửa sổ bù trễ / lịch sử hitbox	200 ms / 1 s	Chặn lạm dụng ping giả
Keep-alive / timeout UDP	1 s / 10 s	
Heartbeat MSP client / game server	15 s	Phát hiện kết nối TCP nửa mở

2.6. Phân loại đối tượng đồng bộ

Loại	Ví dụ	Cách xử lý
Actor được đồng bộ	Người chơi, bot	Có actorId u16, có trong snapshot, mã hoá delta
Phương tiện	Xe, trực thăng, tàu	Luồng riêng S_VEHICLE_SNAPSHOT
Chỉ ở server	Lịch sử hitbox, dữ liệu AI	Không bao giờ gửi
Chỉ ở client	Camera, UI, particle, ragdoll, âm thanh	Không bao giờ gửi
Sự kiện một lần	Chết, nổ, phát súng, chiếm điểm	Kênh tin cậy có thứ tự
Tĩnh	Địa hình, công trình, điểm hồi sinh	Có sẵn trong scene hai phía

2.7. Bảo mật ở mức phù hợp phạm vi

Mối đe dọa	Biện pháp	Tầng
Tăng tốc, dịch chuyển, bắn nhanh, đạn vô hạn	Server chỉ nhận input; kẹp tốc độ; tự quản lý cooldown và đạn	Replication
Mạo danh người chơi	connectionId gắn với playerId trong joinTicket đã ký	Transport
Khuếch đại giả mạo IP	Bắt tay challenge không lưu trạng thái (cookie HMAC)	Transport
Gói rác, lụt gói	Kiểm protocolId; giới hạn tốc độ theo IP; giới hạn nhóm phân mảnh	Transport
Nghe lén mật khẩu	TLS và ghim vân tay chứng chỉ; SHA-256 phía client, bcrypt phía server	Master
Tràn bộ nhớ, Slowloris	Khung tối đa 64 KB; hạn 30 s trước xác thực; 5 kết nối/IP	Master
SQL injection, dò mật khẩu	Truy vấn tham số hoá; 5 lần/phút/IP; khoá 15 phút sau 10 lần sai	Master

2.8. Các quyết định kiến trúc đã chốt

Mã	Quyết định	Lý do
AD-1	Server nắm quyền, không có listen-server	Thảm quyền rõ ràng; tránh sửa 21 singleton
AD-2	Game server là Unity headless	Tái dùng PhysX, A*, AI; tiết kiệm 8-12 tuần
AD-3	Không theo đuổi tắt định (lockstep)	Bộ sinh ngẫu nhiên rải rác; engine vật lý không tắt định
AD-4	Ragdoll chỉ là hiệu ứng cục bộ	Tiết kiệm khoảng 1,7 MB/s
AD-5	Transport, Replication là thư viện .NET thuần	Kiểm thử bằng xUnit, tránh xung đột Unity
AD-7	Snapshot không tin cậy, sự kiện tin cậy	Đúng bản chất trạng thái và sự kiện
AD-8	Không dùng WebSocket	Yêu cầu môn học và head-of-line blocking

2.9. Quy trình phát triển và bảo đảm chất lượng

- **Đặc tả giao thức đóng băng** là nguồn sự thật duy nhất của hệ thống; công cụ kiểm tra tự động sẽ cảnh báo nếu cấu trúc gói tin của các bên không đồng nhất.
- **Tách người thiết kế và người kiểm thử:** nhóm giao vận xây dựng cơ chế đóng gói dữ liệu, nhóm đồng bộ xây dựng bộ kiểm thử tính tương thích theo đặc tả.
- **Tích hợp liên tục (CI)** chạy tự động: kiểm tra toàn diện, coi cảnh báo là lỗi, chạy toàn bộ bộ kịch bản kiểm thử.

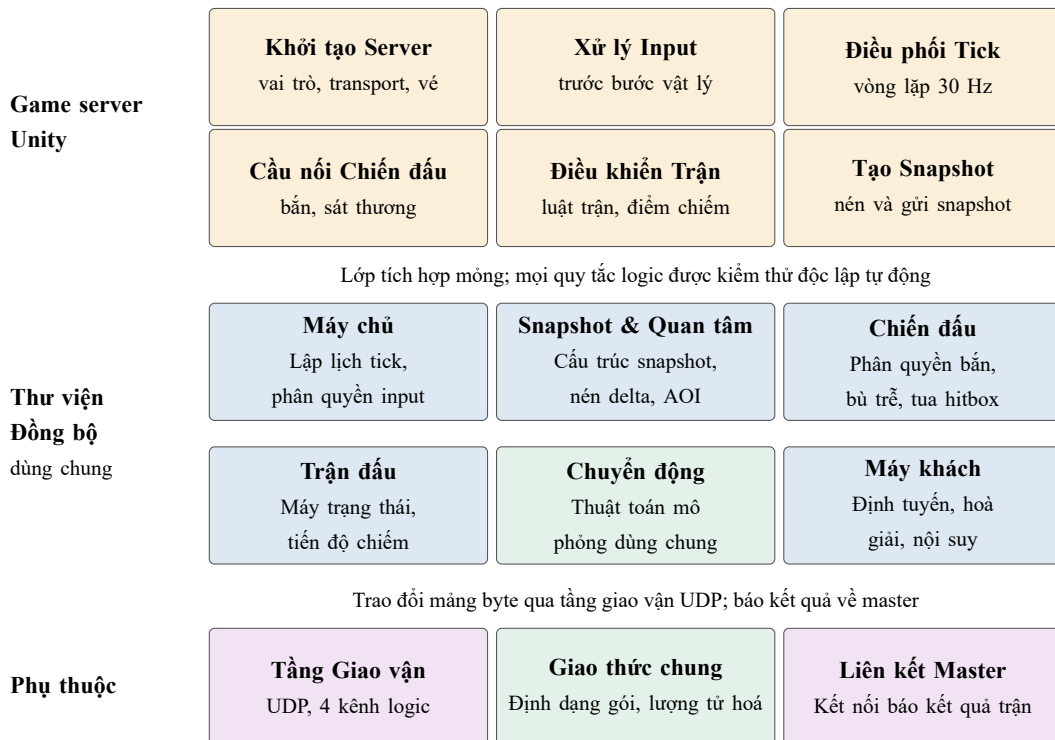
- **Tiêu chuẩn hoàn thành (Definition of Done):** hoàn thành kiểm thử thực tế, toàn bộ kiểm nghiệm tự động đạt chuẩn, ứng dụng biên dịch sạch sẽ, tài liệu kỹ thuật hoàn thiện.

II. NỘI DUNG CÁ NHÂN - Phân hệ đồng bộ trạng thái và mô phỏng server (Replication).

3. Biểu đồ và giao diện nội dung cá nhân thực hiện

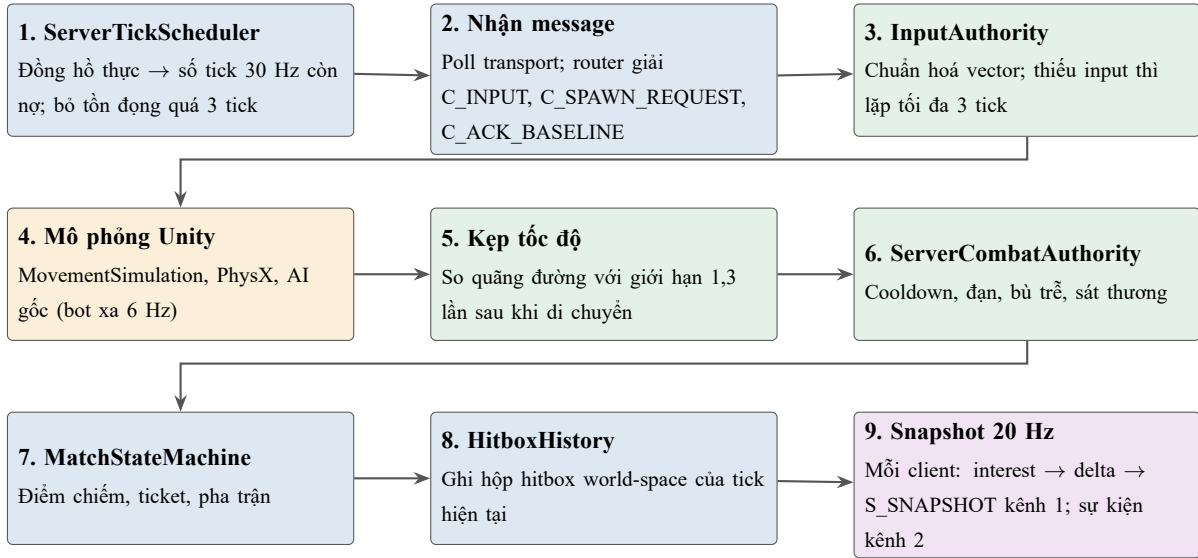
Phân hệ Replication đứng giữa: **nhận byte từ Transport và biến thành trạng thái trò chơi**, và ngược lại. Nhiệm vụ gồm: nén trạng thái thể giới (lượng tử hoá, delta, quản lý vùng quan tâm) để 48 actor vừa ngân sách 8 KB/s; vòng mô phỏng có thẩm quyền 30 Hz chống gian lận; bù trễ khi bắn; thuật toán mô phỏng chuyển động dùng chung client và server; và bộ kịch bản kiểm thử tính tương thích làm trọng tài cho giao thức.

3.1. Kiến trúc phân hệ Replication



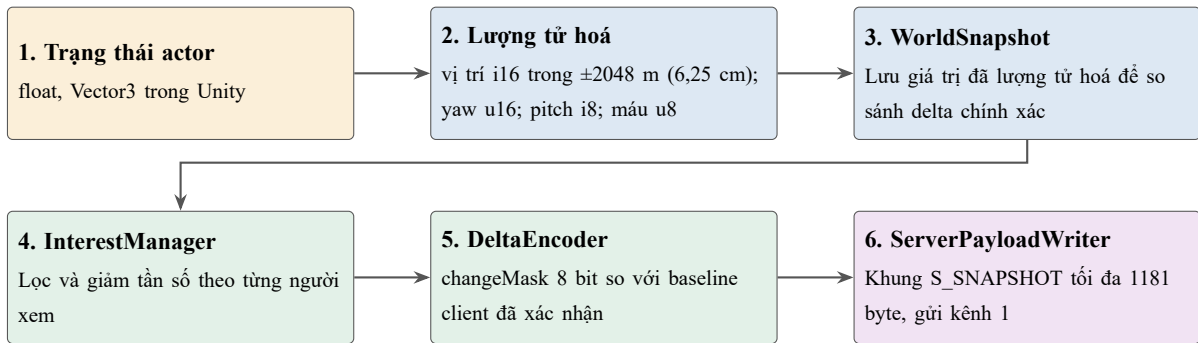
Hình 3.1. Kiến trúc phân hệ: lớp tích hợp Unity mông gọi vào thư viện đồng bộ logic kiểm thử được độc lập

3.2. Vòng tick có thẩm quyền của server



Hình 3.2. Các bước của một tick mô phỏng có thẩm quyền trên game server

3.3. Đường ống nén snapshot

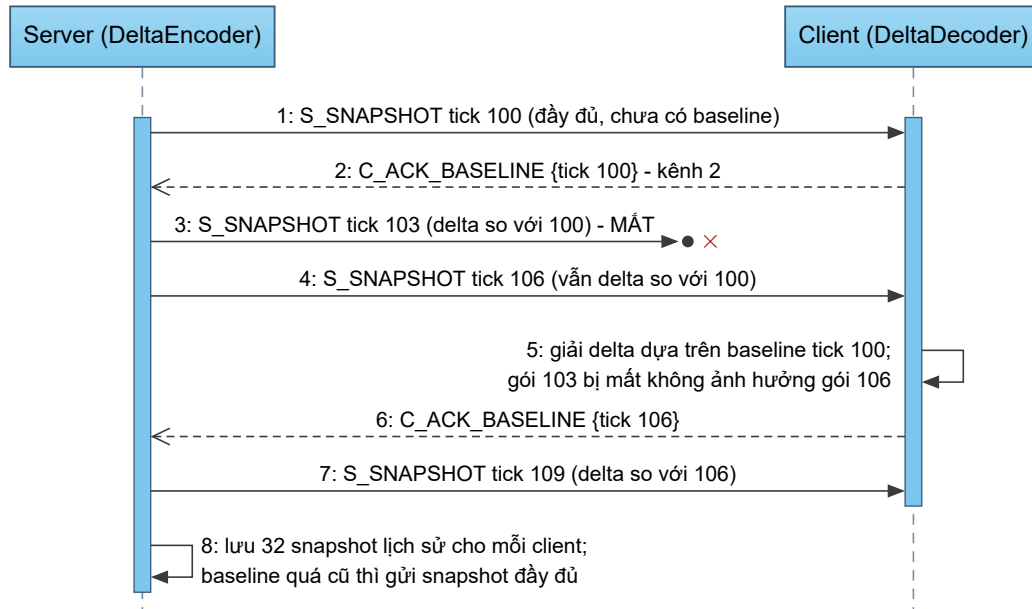


Hình 3.3. Đường ống nén từ trạng thái mô phỏng đến gói S_SNAPSHOT

Cấu hình (16 người + 32 bot, bản đồ Dustbowl, 5 phút)	KB/s mỗi client	Snapshot trung bình	Tiết kiệm lũy kế
Gốc: snapshot đầy đủ, căn byte, không interest	19,86	1017 byte	-
+ bit-packing (đo, không đưa vào bản chạy)	19,04	975 byte	4,1%
+ delta encoding	9,94	509 byte	49,9%
+ quản lý vùng quan tâm	1,50	77 byte	92,4%
Server đã triển khai (định dạng v1 căn byte)	1,67	-	21% ngân sách 8 KB/s

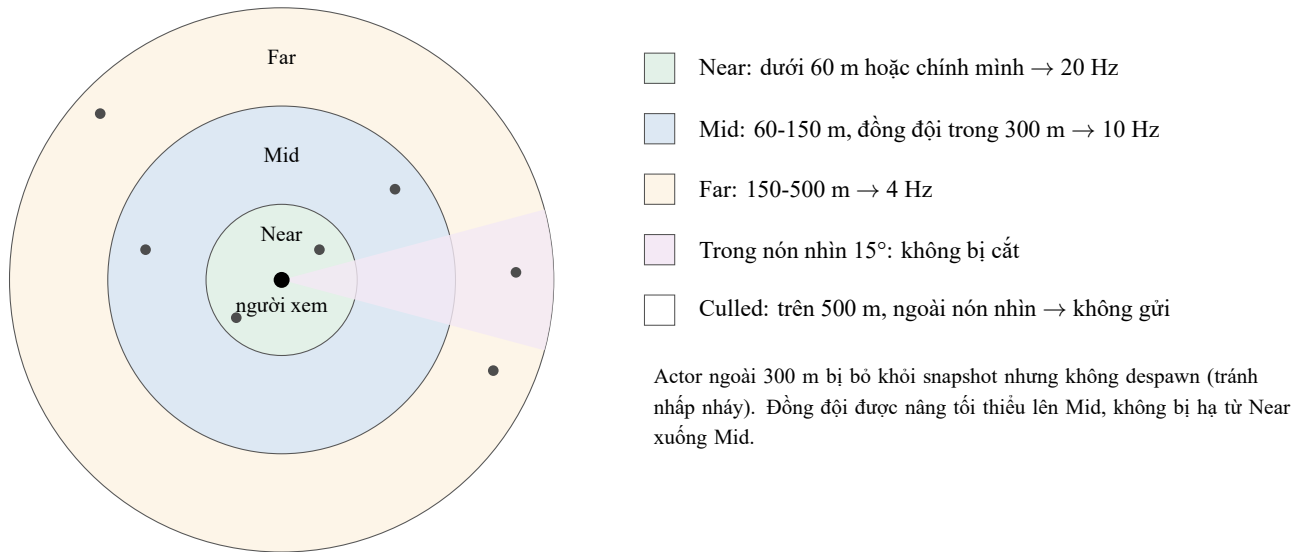
Hình 3.4. Kết quả đo đóng góp của từng kỹ thuật nén (in bởi Phase04ExperimentTests)

3.4. Delta theo baseline đã xác nhận



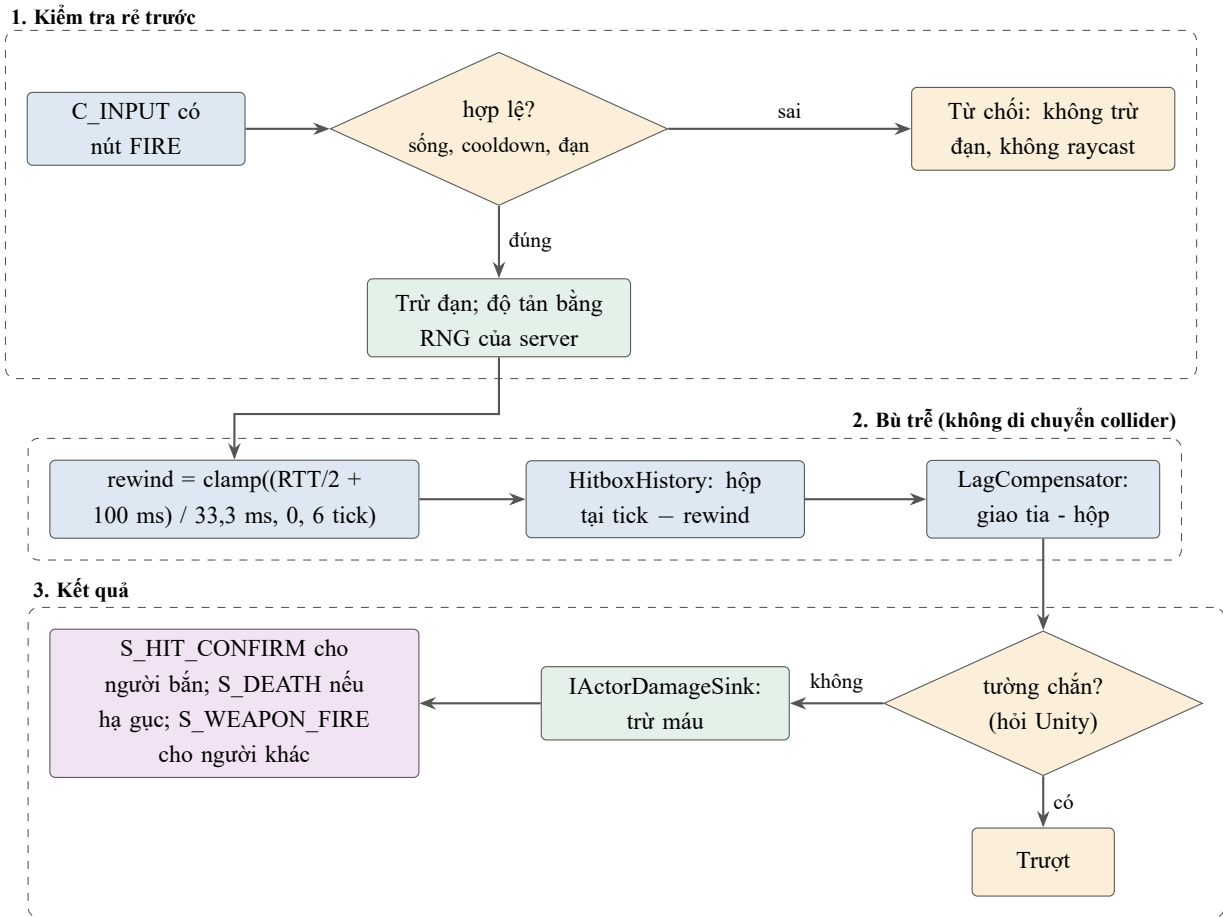
Hình 3.5. Delta chỉ so với baseline client đã xác nhận: một gói mất không làm hỏng chuỗi delta

3.5. Quản lý vùng quan tâm



Hình 3.6. Các vùng quan tâm theo khoảng cách từ người xem và tần số cập nhật (không theo tỉ lệ)

3.6. Bắn có thẩm quyền và bù trễ

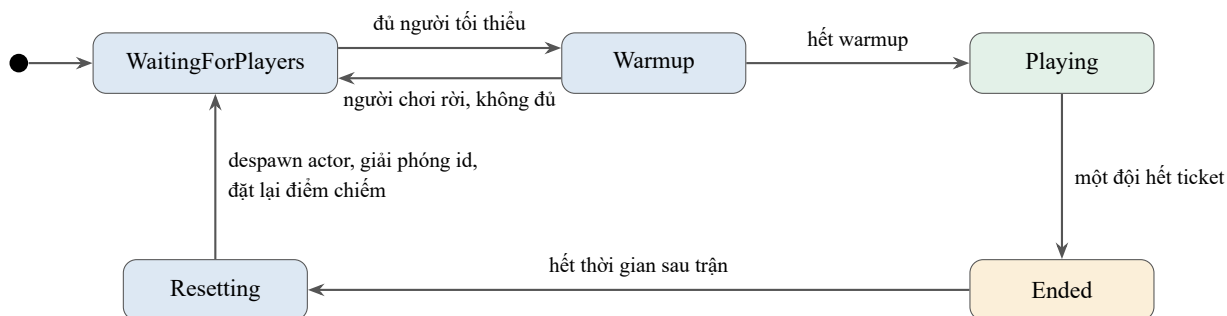


Hình 3.7. Luồng phân xử phát bắn có thẩm quyền với bù trễ tối đa 200 ms

RTT	0 ms	50 ms	100 ms	150 ms	200 ms	300 ms
Có bù trễ	100%	100%	100%	100%	100%	100%
Không bù trễ	0%	0%	0%	0%	0%	0%

Hình 3.8. Tỷ lệ trúng mục tiêu chạy ngang 5 m/s theo RTT (mục tiêu đứng yên: 20/20 trong cả hai trường hợp)

3.7. Máy trạng thái trận đấu

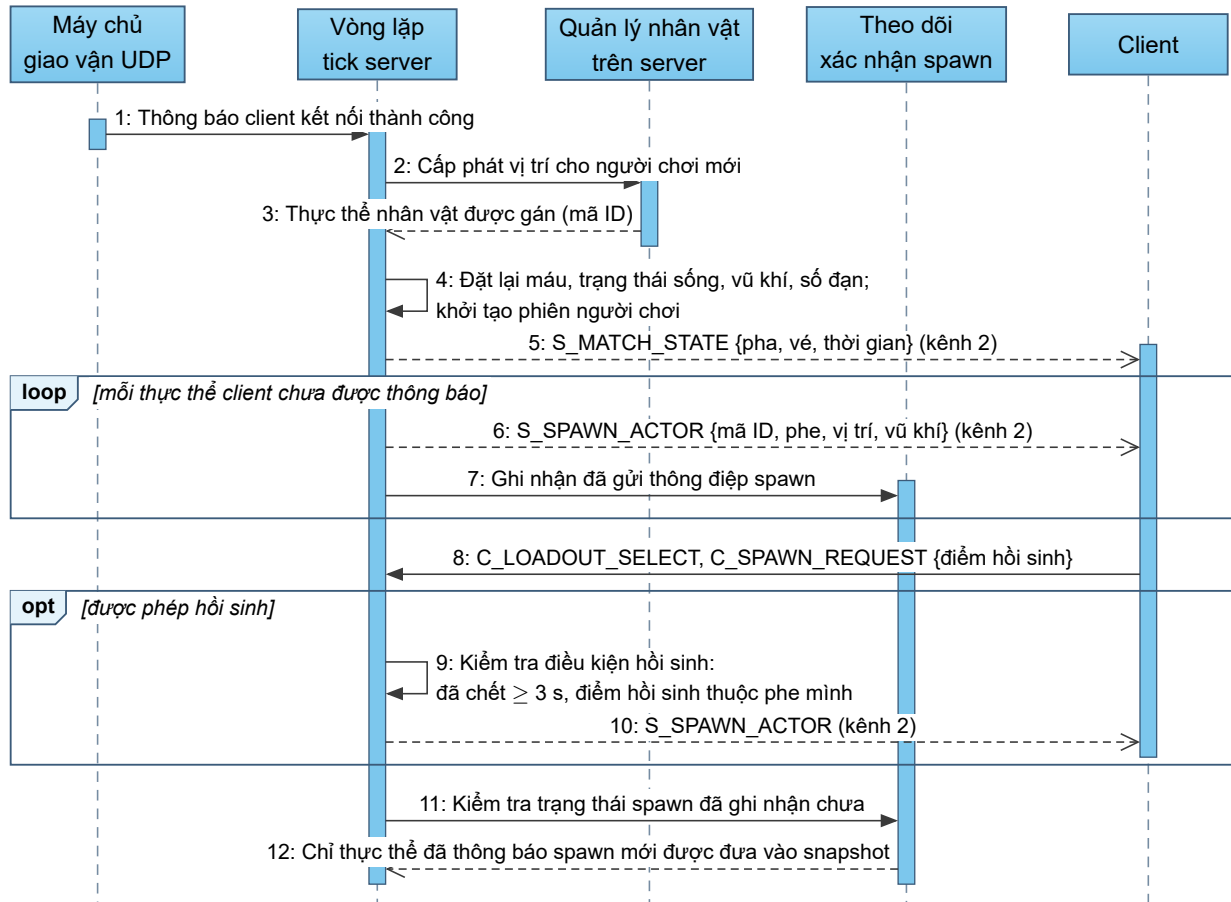


Hình 3.9. MatchStateMachine - pha trận được phát tới client qua `S_MATCH_STATE`

4. Luồng xử lý và giải pháp đã ứng dụng

Mục này trình bày chi tiết các luồng nghiệp vụ mà phân hệ đồng bộ và mô phỏng server hiện thực: người chơi vào trận và được tạo thực thể, input đi vào vòng tick có thẩm quyền, snapshot được nén và gửi riêng cho từng client, phát bắn được phân xử với cơ chế bù trễ, trận đấu diễn tiến qua các pha và báo kết quả về master.

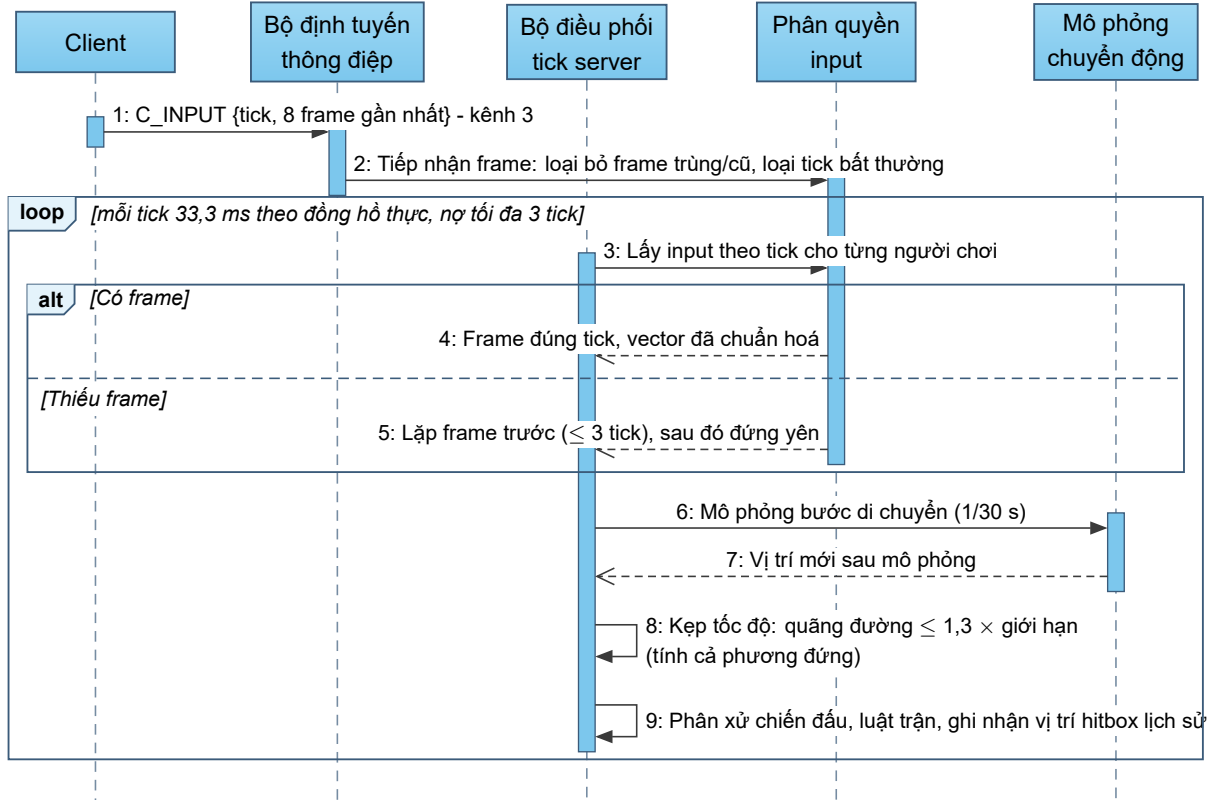
4.1. Luồng người chơi vào trận và spawn



Hình 4.1. Biểu đồ tuần tự người chơi vào trận, chọn trang bị và hồi sinh

Vấn đề	Giải pháp đã áp dụng
Snapshot vượt trước gói tạo nhân vật do hai kênh không ràng buộc thứ tự	Sắp thứ tự ở phía gửi: giữ thực thể khỏi snapshot của một client cho tới khi thông điệp tạo nhân vật đó đã được đưa cho giao vận; kênh tin cậy bảo đảm gói tạo tới trước
Người vào sau nhận trạng thái cũ của người vừa rời	Khi cấp phát vị trí, đặt lại chỉ số máu, trạng thái sống, vũ khí và băng đạn trước snapshot đầu tiên
Client gian lận bỏ qua thời gian chờ hồi sinh	Server là nơi nắm quyền quyết định, áp dụng cùng tham số thời gian chờ hồi sinh với đồng hồ đếm ngược phía client
Mã ID nhân vật bị tái cấp phát khi snapshot cũ còn trên đường truyền	Giữ mã ID đã thu hồi trong vùng cách ly 5 s trước khi cấp lại; luôn đảm bảo còn đủ ID rảnh

4.2. Luồng input và vòng tick có thẩm quyền

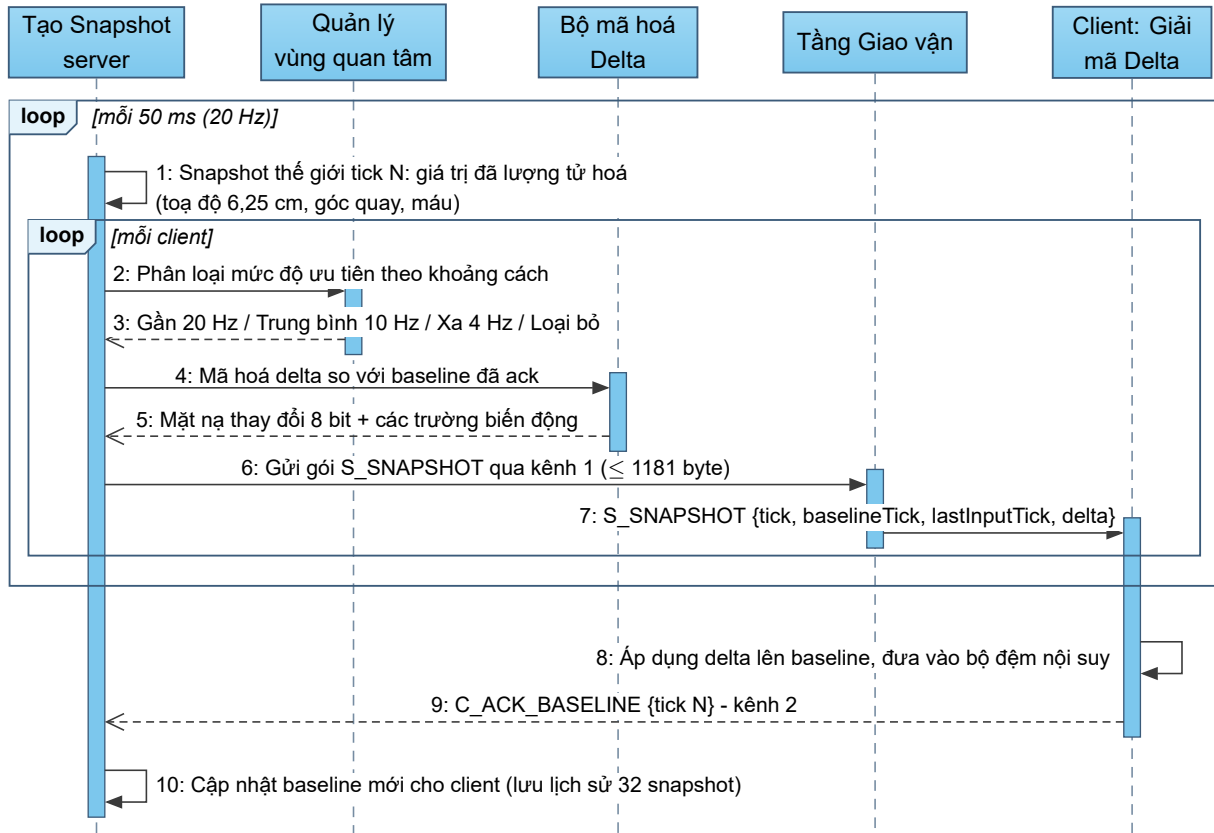


Hình 4.2. Biểu đồ tuần tự nhận input và chạy vòng tick 30 Hz có thẩm quyền

Vấn đề	Giải pháp đã áp dụng
Tick mạng phụ thuộc chu kỳ vật lý của đồ họa	Tính tick theo đồng hồ thực với bộ tích lũy thời gian, hoàn toàn độc lập với tốc độ khung hình
Server chậm một lần rồi không bao giờ đuổi kịp	Giới hạn nợ tối đa 3 tick; phần thời gian thừa bị bỏ qua và ghi nhận cảnh báo
Mất gói input	Client gửi lặp các frame gần nhất trong mỗi gói; server thiếu frame thì lặp frame trước tối đa 3 tick (100 ms) rồi dừng
Gian lận tốc độ di chuyển	Chuẩn hoá vector di chuyển; sau khi di chuyển so sánh quãng đường với giới hạn tốc độ tối đa
Tick giả làm hỏng vòng đệm	Loại bỏ các frame có chỉ số tick nhảy quá 60 tick (2 s) so với frame gần nhất

Đo: client gửi giá trị trực tối đa của 18 trên cả hai trục không đi xa hơn người chạy nước rút trung thực.

4.3. Luồng tạo, nén và gửi snapshot

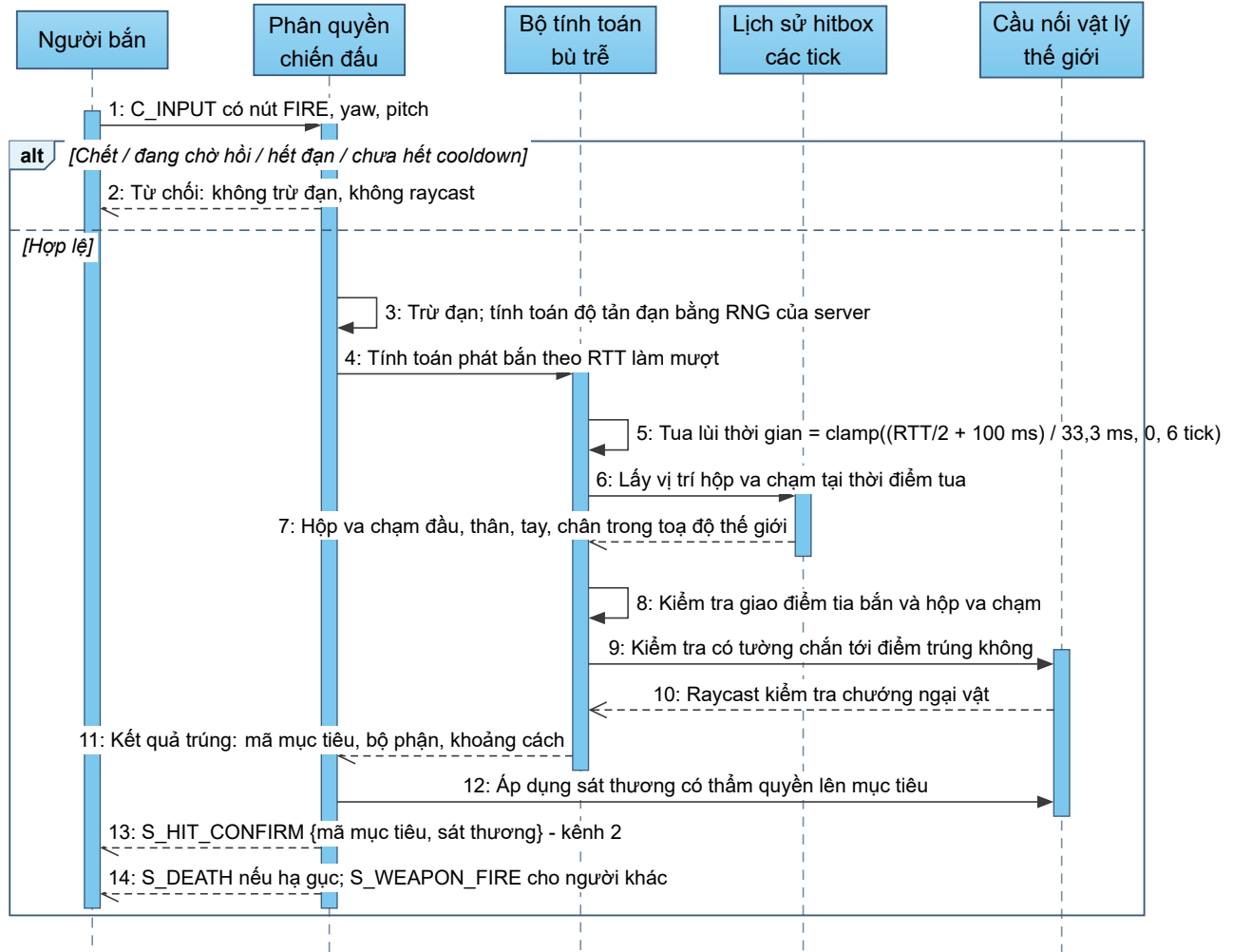


Hình 4.3. Biểu đồ tuần tự đường ống snapshot: lượng tử hoá, vùng quan tâm, delta theo baseline đã xác nhận

Giải pháp và lý do.

- **Lưu giá trị đã lượng tử hoá trong WorldSnapshot.** Nếu lưu float, vật lý làm vị trí của actor đứng yên rung ở chữ số thập phân thứ mười nên phép so “có thay đổi không” luôn trả lời có và delta không tiết kiệm được gì.
- **Delta so với baseline client đã xác nhận, không so với snapshot liền trước.** Delta liền trước là nhỏ nhất nhưng mất một gói là mọi snapshot sau đều không giải được. Với baseline đã ack (C_ACK_BASELINE qua kênh tin cậy), mất một snapshot chỉ mất đúng gói đó. Chi phí: 32 snapshot lịch sử mỗi client. Kiểm thử: mất ngẫu nhiên 20% gói trong 1 000 tick, trạng thái cuối khớp chính xác với 4 hạt giống.
- **Vùng quan tâm theo người xem.** Near < 60 m hoặc chính mình: 20 Hz; Mid 60-150 m hoặc đồng đội trong 300 m: 10 Hz; Far 150-500 m: 4 Hz; ngoài 500 m và ngoài nón nhìn 15°: không gửi. Đồng đội được nâng **tối thiểu** lên Mid chứ không bị hạ từ Near xuống Mid. Actor xa bị bỏ khỏi snapshot nhưng không despawn để không nhấp nháy.
- **Kết quả.** 16 người + 32 bot trên Dustbowl: từ 19,86 KB/s xuống 1,50 KB/s mỗi client (Hình 3.4); server đã triển khai đo 1,67 KB/s, bằng 21% ngân sách 8 KB/s.

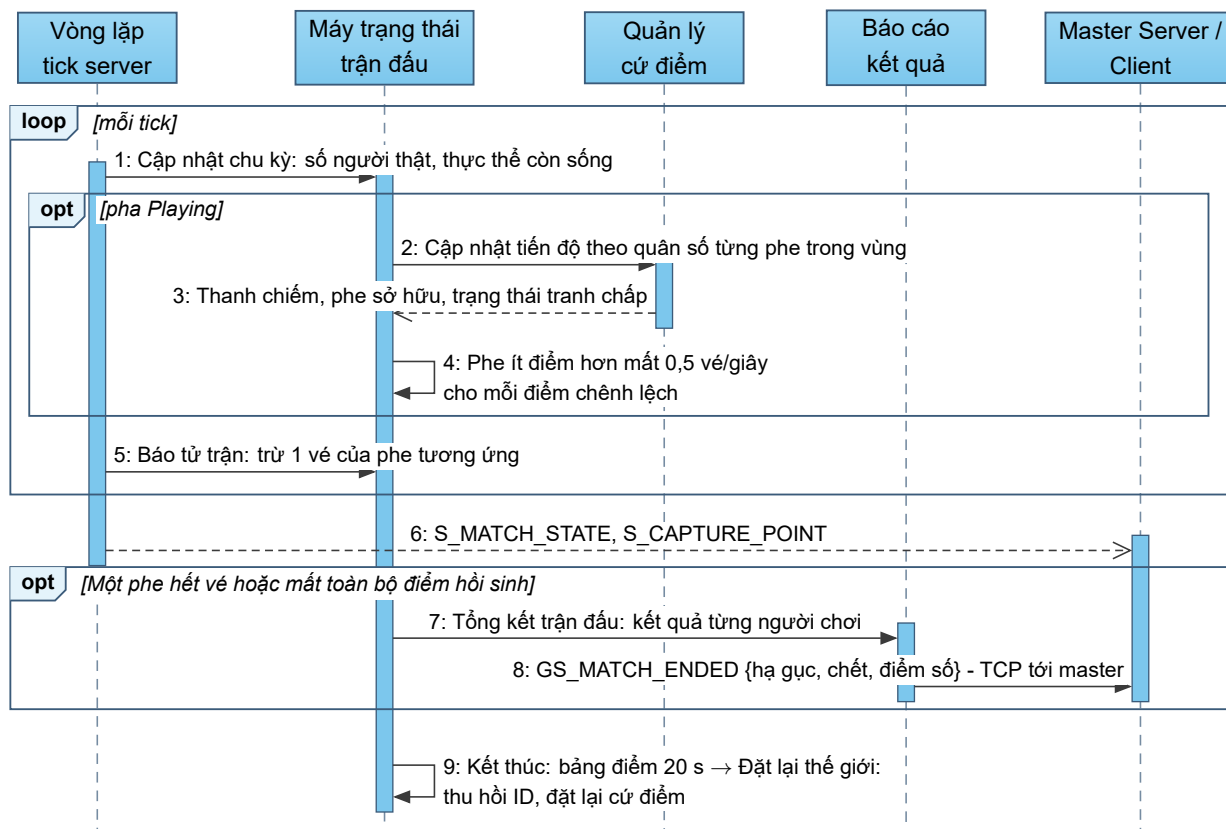
4.4. Luồng phân xử phát bắn với bù trễ



Hình 4.4. Biểu đồ tuần tự phân xử phát bắn: kiểm tra rẽ trước, tua hitbox bằng dữ liệu lịch sử, kiểm tra tường chắn

Vấn đề	Giải pháp đã áp dụng
Bắn nhanh / đạn vô hạn	Server tự quản lý cooldown và số đạn; kiểm tra còn sống, cooldown, đạn trước mọi phép toán kém: 100 lệnh bắn trong 1 s với cooldown 0,1 s chỉ 10 lệnh được xử lý, 90 lệnh bị từ chối không trừ đạn, không raycast
Người chơi nhìn thấy quá khứ (trễ mạng + 100 ms nội suy) nên bắn trúng mục tiêu đang chạy luôn trượt	Lưu 1 s hộp va chạm (30 tick × 4 hộp mỗi actor) và kiểm tra tia với hộp tại thời điểm người bắn nhìn thấy; mục tiêu chạy ngang 5 m/s: 100% trúng với mọi RTT 0-300 ms, không bù trễ: 0%
Tua bằng cách dời collider rồi trả về để để lại hitbox “kẹt trong quá khứ” khi có ngoại lệ	Lịch sử lưu hộp toạ độ thế giới : tua là đọc một giá trị, không di chuyển gì; chỉ câu hỏi “có tường không” mới gọi vào Unity
Khai RTT giả để được tua xa	Dùng RTT server tự đo, kẹp tối đa 6 tick (200 ms)
Actor vừa trở nên liên quan chưa có lịch sử	Dùng tư thế hiện tại, gán cờ trong kết quả và tăng bộ đếm (không im lặng)

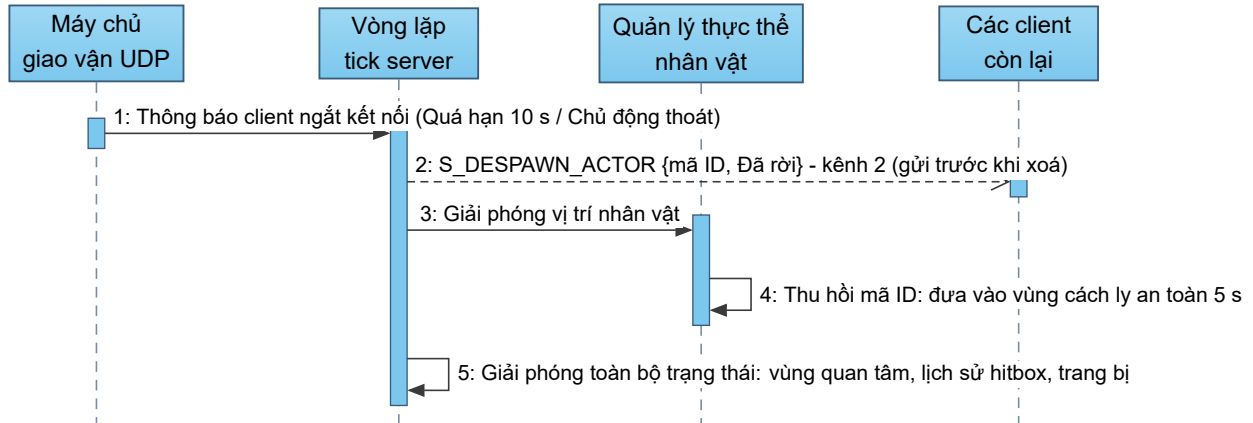
4.5. Luồng trận đấu, điểm chiếm cứ và báo kết quả



Hình 4.5. Biểu đồ tuần tự luật trận: tick, điểm chiếm cứ, trừ vé, kết thúc và báo kết quả về master

Luật trong đề	Hiện thực kỹ thuật và tham số
Năm pha: chờ người, khởi động, đang đánh, kết thúc, dọn dẹp	Chờ người chơi → Khởi động → Đang thi đấu → Kết thúc → Đặt lại thế giới (Hình 3.9)
Ít nhất 2 người thật, bot không tính; đếm ngược 20 s	Tối thiểu 2 người chơi thật để bắt đầu; thời gian khởi động 20 s; người rời làm thiếu người thì quay về chờ
Mỗi phe 200 vé, mỗi lần chết mất 1 vé	200 vé ban đầu cho mỗi phe; trừ 1 vé khi người chơi hoặc bot tử trận
Phe giữ ít điểm hơn mất 0,5 vé/giây cho mỗi điểm chênh	Trừ 0,5 vé/giây cho mỗi cứ điểm chênh lệch; bảng điểm thì không bị trừ
Tốc độ chiếm theo chênh lệch quân số, chỉ tới 4 người; bằng người thì tranh chấp	Tối đa 4 người tính điểm chênh lệch; chênh lệch 0 thì tranh chấp; chỉ gửi gói tin khi thanh đổi quá 2%
Mất toàn bộ điểm hồi sinh thì thua	Kiểm tra điểm hồi sinh mỗi phe; luật chỉ áp dụng sau 1 s đầu vòng để tránh xử hoà ngay tick đầu
Hết trận hiện bảng điểm 20 s rồi chơi tiếp	Thời gian bảng điểm cuối trận 20 s; sau đó tự động đặt lại cứ điểm và vé mà không cần thoát phòng

4.6. Luồng người chơi rời trận



Hình 4.6. Biểu đồ tuần tự rời trận: thông báo cho người còn lại trước, rồi mới giải phóng tài nguyên

Thứ tự gửi S_DESPAWN_ACTOR trước khi xoá bảng là có chủ đích: nếu xoá trước, người còn lại thấy một “hình nộm” đứng bất động, vẫn bắn trúng được, ở vị trí cuối của người đã rời cho tới hết trận. Transport coi client im lặng quá 10 s là rời trận và luồng này xử lý y hệt trường hợp chủ động thoát, đúng yêu cầu trong đề.

4.7. Bảng ánh xạ hình vẽ với chức năng cá nhân thực hiện

Bảng tổng hợp đối chiếu toàn bộ các sơ đồ kiến trúc, máy trạng thái, cơ chế thuật toán và luồng xử lý do cá nhân thực hiện tương ứng với các chức năng cụ thể trong phân hệ đồng bộ trạng thái và mô phỏng server:

Hình vẽ	Tên hình vẽ / biểu đồ	Chức năng cá nhân thực hiện
Hình 3.1	Kiến trúc phân hệ Replication	Thiết kế phân tầng phân hệ đồng bộ: tách biệt lớp logic .NET thuần (kiểm thử độc lập) với lớp tích hợp engine trên game server.
Hình 3.2	Vòng tick có thẩm quyền	Hiện thực vòng lặp mô phỏng vật lý và trạng thái 30 Hz (33,3 ms) độc lập với FPS đồ họa; kiểm soát nợ tick chống hiện tượng vòng xoáy tử thần.
Hình 3.3	Cấu trúc nén trạng thái thực thể	Thiết kế cấu trúc lưu trữ snapshot thế giới nhị phân (tối đa 48 thực thể) và thuật toán phân loại tần suất gửi theo vùng quan tâm (Near/Mid/Far).
Hình 3.4	Tối ưu hoá băng thông mạng	Kết quả đo lường và tối ưu hoá: giảm băng thông truyền từ 19,86 KB/s xuống còn 1,50 KB/s mỗi client, đáp ứng xuất sắc giới hạn thiết kế 8 KB/s.
Hình 3.5	Cơ chế mã hoá vi sai Delta	Hiện thực kỹ thuật nén delta so với baseline đã xác nhận: chỉ gửi mặt nạ thay đổi (changeMask 8-bit) và dữ liệu có biến động thực tế.
Hình 3.6	Lọc vùng quan tâm Area of Interest	Phân vùng không gian và góc nhìn camera để lọc thực thể liên quan, loại bỏ truyền dữ liệu thừa cho các thực thể ngoài tầm quan sát.
Hình 3.7	Cơ chế bù trễ phát bắn	Hiện thực giải thuật tua ngược hộp va chạm (hitbox history) về đúng thời điểm người bắn nhìn thấy trong quá khứ, giải quyết triệt để vấn đề trễ mạng.
Hình 3.8	Tỷ lệ nén dữ liệu theo kịch bản	Đánh giá hiệu quả thuật toán nén snapshot trong các tình huống thực chiến: di chuyển tự do, giao tranh cụm và kịch bản tĩnh.
Hình 3.9	Máy trạng thái trận đấu	Thiết kế máy trạng thái vòng đời trận đấu gồm 5 pha: Chờ người chơi, Khởi động, Đang thi đấu, Kết thúc và Dọn dẹp/đặt lại thế giới.
Hình 4.1	Sơ đồ tuần tự vào trận	Hiện thực luồng tiếp nhận người chơi, cấp phát vị trí thực thể, đồng bộ trạng thái trận và kiểm soát thời gian hồi sinh an toàn.
Hình 4.2	Sơ đồ tuần tự vòng tick	Hiện thực luồng thu thập input từ client, xác thực tính hợp lệ, chuẩn hoá vector, kẹp tốc độ và thực thi bước mô phỏng chuyển động có thẩm quyền.
Hình 4.3	Sơ đồ tuần tự tạo snapshot	Hiện thực luồng lượng tử hoá dữ liệu, lọc vùng quan tâm theo khoảng cách/góc nhìn và đóng gói snapshot nén gửi qua kênh UDP có thứ tự.
Hình 4.4	Sơ đồ tuần tự phân xử phát bắn	Hiện thực quy trình kiểm tra điều kiện bắn (cooldown, số đạn), tua ngược hitbox lịch sử, kiểm tra va chạm tia và áp dụng sát thương có thẩm quyền.
Hình 4.5	Sơ đồ tuần tự luật trận	Hiện thực cập nhật tiến độ chiếm cứ điểm theo quân số, cơ chế trừ vé tự động, phân định thắng thua và báo cáo tổng kết kết quả về máy chủ trung tâm.
Hình 4.6	Sơ đồ tuần tự rời trận	Hiện thực luồng thông báo hủy thực thể (despawn) cho các người chơi khác trước khi thu hồi mã định danh và giải phóng tài nguyên.